

Exercise #1 - Familiarity with Linux OpenCV

Mason McGaffin

Exercise #1

ECEN 5763 Embedded Computer Vision

University of Colorado Boulder

June 05, 2026

Contents

1 EXPLORE OPENCV	3
1.1 SUMMARY.....	3
1.2 CODE DIFFERENCES ACROSS OPENCV VERSIONS	3
2 TESTING OPENCV ON THE JETSON ORIN NANO.....	3
2.1 BRIGHTEN	3
2.2 CAPTURE.....	5
2.3 DIFFCAPTURE.....	6
3 CONVOLUTION.....	7
3.1 SHARPEN-PSF	7
3.2 MODIFICATIONS	9
3.3 SHARPEN-GRID.....	10
4 FACE DETECTION	11
5 ANNOTATED IMAGE	13
6 REFERENCES.....	14
7 APPENDIX	15

1 Explore OpenCV

Source: [Explore video analytics in the cloud](#)

1.1 Summary

The article, published in 2013 by Dr. Sam Siewert, begins by framing video analytics in cloud-scaled computing as a data-centric (rather than compute-centric) problem. The article introduces the digital media formats and developer tools of the video analytics toolchain including OpenCV, FFmpeg, and GIMP. The author then defines several disciplines of video analytics, explaining that the field encompasses computer vision as well as other disciplines such as image processing and machine learning. The author further discusses video analytics, dividing such systems into embedded on-device intelligent sensors and cloud-based processing systems based on their application. The article focuses particularly on augmented reality and skeletal transformation. The author highlights the improved interactive experience and enhanced safety features of augmented reality and the impacts of skeletal transformation for health and safety applications used in computer vision. Finally, the article argues that in the future, video analytics will impact many environments of the everyday world and will be fundamental to developing intelligent computing systems.

1.2 Code differences across OpenCV versions

The first apparent difference is that in OpenCV 4.x is that it is now requiring C++ instead of allowing some C code like versions 3.x and 2.x did. The entire C API has been removed. The library modules have been restructured to be flattened. Also, a new module is included, *imgcodecs*. This contains the *imread/imwrite* functions which were separated out of the *highgui* module. Another significant change is that previous C macros were replaced by C++ enums.

From these detections, we can assume that other changes were made involving modularity improvements, API changes, and the removal of old C code. Given recent improvements in artificial intelligence and machine learning, it is probable that new features/modules have been added to support advanced processing.

2 Testing OpenCV on the Jetson Orin Nano

2.1 brighten

The `brighten` executable takes one parameter, the input image file, and then accepts two inputs, alpha brighten factor and bias contrast increase value, from the user via the terminal (stdin) to modify the original image. The program applies the operation in Equation (1) for each color value, *c*, of each pixel of *image* to create *new_image*. Once *new_image* is created, the executable creates two named windows to display the original and modified images side by side as seen in Figure 2.1.

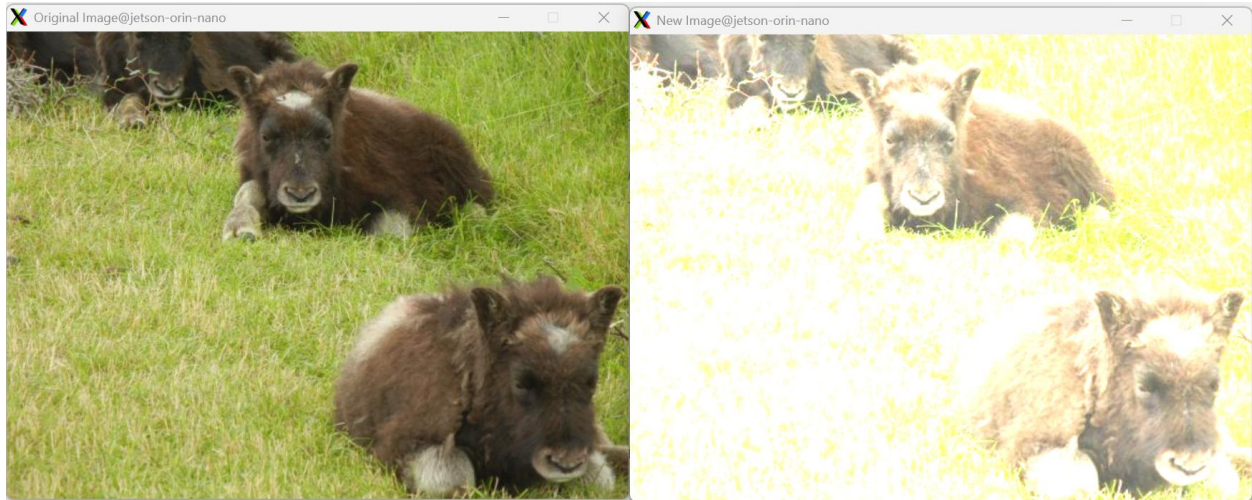


Figure 2.1 Brighten application example – $\alpha=2.0$; $\beta=75$

$$c'_i = \alpha \times c_i + \beta \quad (1)$$

This starter code could be the starting point for a variety of apps including a photo editing application. The developer could implement a GUI and more editing skills/features for an effective photo editing application. Another application that could be built based on this starter code, is a medical imaging application. The contrast and brightness adjustments could be used in X-ray and CT scans to target specific bone and tissue densities.

Function descriptions:

1. imread()
 - a. This function loads the image from the specified file and as a 'Mat' object, which is an n-dimensional array used, in this case, for the color JPEG photo provided.
2. saturate_cast()
 - a. This function ensures the pixel manipulation is safe and accurate. It prevents overflow, underflow, and corruption. The OpenCV template function differs from the standard C++ static_cast because it clamps values at the minimum and maximum boundaries as opposed to wrapping.
3. imshow()
 - a. This function displays the image object in the specified window. In our case, the image is displayed at its original size, as the window size is not specified. However, the image could be scaled to fit the window, as constrained by the screen resolution.

2.2 capture

The capture application opens the video camera connected to our embedded machine and displays the video in a window. The camera displays each 640x480 pixel frame in a window of the same size to have near real-time video display. If the user presses the 'ESC' key, they can close the application. If the user presses the 'n' key, the application will log it in stdout. See Figure 2.2.



Figure 2.2 Capture application windows

The starter code could be used to build a video conferencing application. The developer would need to display the frame to both the sender and receiver among the other aspects of the application including socket connections, starting and ending a video call, webchat, security, etc.

Function descriptions:

1. `cam0.isOpened()`
 - a. This OpenCV library function determines if there is a device connected to the machine that is already initialized for video capturing. In our case, we are using the Logitech C920x USB Webcam.
2. `cam0.set()`
 - a. This function is used to configure the video capturing device. In the capture application, the horizontal and vertical resolution are configured to 640x480 pixels using this function.
3. `cam0.read()`

- a. This function reads the current frame of the video captured by our webcam into a 'Mat' object array. The function stores each pixel value in the object which is later used in our application to update the generated window.

2.3 diffcapture

The diffcapture application is similar to the capture application except it goes one or two steps further. The application starts out similarly, opening the webcam device, configuring it to be 640x480 pixel resolution. However, after reading the frame, the picture is transformed into a grayscale. Then, the grayscale frame is compared to the previous frame to create a *mat_diff* frame and calculate a percentage of the pixels that differ. Finally, the application updates three generated windows to show the current grayscale frame, the previous grayscale frame, and the current *mat_diff* frame. As seen in Figure 2.3, I was able to shake the web camera in order to simulate movement of the object in the frame. The *mat_diff* frame, highlights, in grayscale, all pixels which are changing value indicating motion. This effectively outlines any object that is moving in front of the camera. To exit the application, the user presses the 'q' key.



Figure 2.3 Diffcapture application windows

The starter code could be used to build a trail camera. The pixel difference tracked could be used to identify when wildlife or game enters the camera frame for viewing.

Function descriptions:

1. `cv::cvtColor()`
 - a. This function is used to modify the color format of a frame object. This can be used to convert between color spaces, families and perceptions and add the alpha channel. In our case, it is used to convert the frame to grayscale.
2. `absdiff()`
 - a. We utilize this OpenCV function in diffcapture to compute the absolute difference of each pixel between two frames and output that difference to a new frame. The function compares each pixel value and stores the difference in the output frame. The output *mat_diff* frame highlights changing pixels based on the current and previous frames.
3. `Cv::putText()`

- a. This function displays the desired text on the frame image specified. This is done before updating the window with the new frame. Our use case of this function is to display the percent difference of all of the pixels on the “Gray Diff” window.

3 Convolution

3.1 sharpen-psf

The sharpen application processes images pixel by pixel. It was modified to dynamically read the image header to determine width and height of the image. Then, it uses variable length arrays to allocate memory for the image pixels. Sharpen is a single threaded application. It processes the pixels sequentially from top-left to bottom-right. The core functionality of sharpening the image comes from applying the PSF filter, a 2D convolution matrix, across the pixels. This filter compares the pixels around it to identify sudden changes in pixel brightness. These changes over short distances indicate edges or lines in the image. The function uses this to amplify the edges observed. See the examples below used on the Trees.jpg image, shown in Figure 3.1.



Figure 3.1 Original Image - Trees.jpg

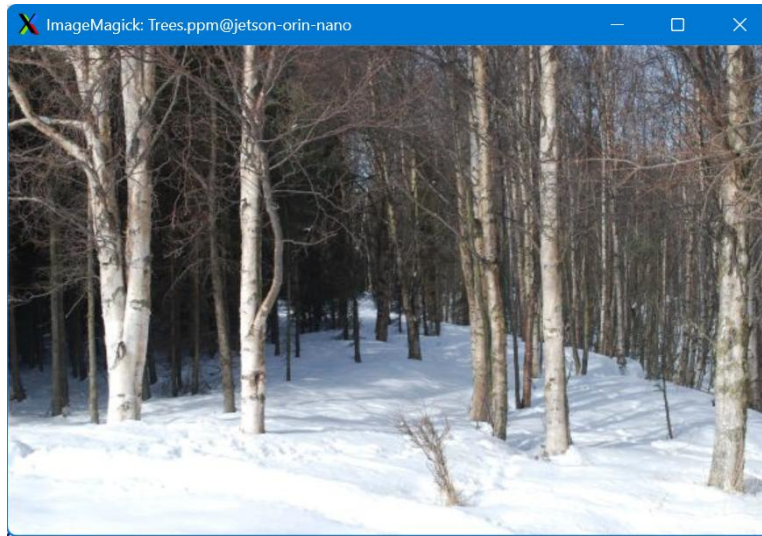


Figure 3.2 Converted to PPM – Trees.ppm

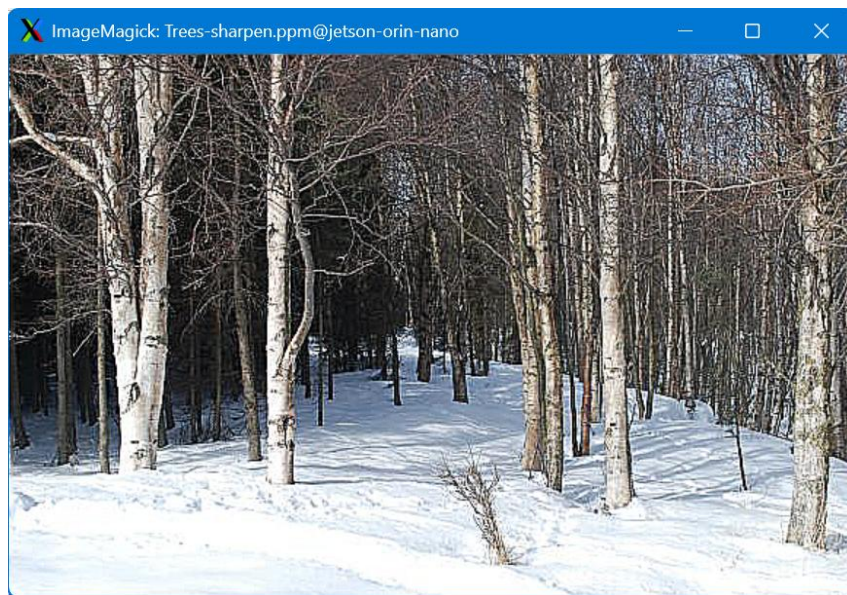


Figure 3.3 Sharped – Trees-sharpen.ppm

As you can see when comparing Figure 3.2 and Figure 3.3, the sharpen application boldens the outline of the tree in the image. It also emphasizes the changes in the snow and overall makes the image look more vibrant.

Another example is below of the sharpen tool used on the Cactus image provided.

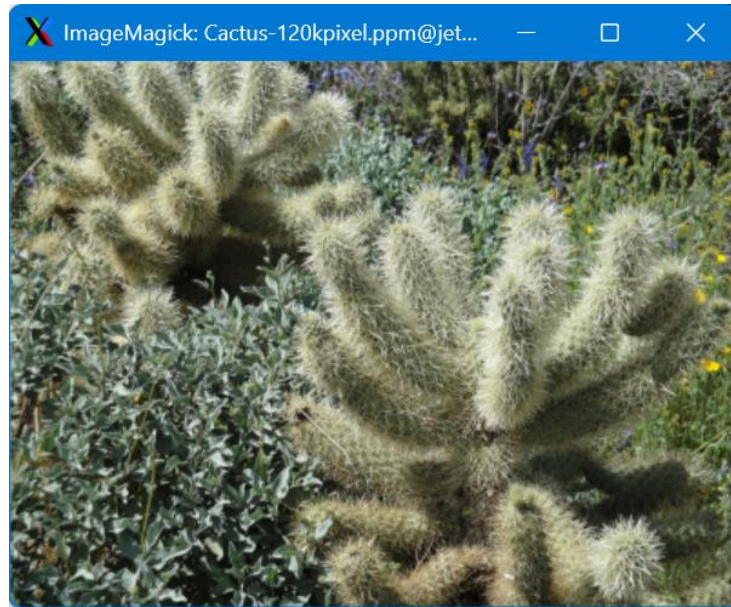


Figure 3.4 Original – Cactus-120kpixel.ppm

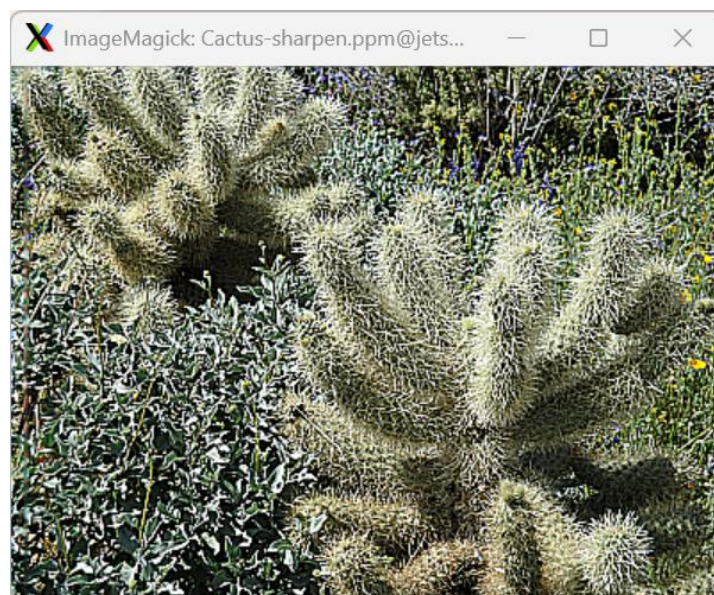


Figure 3.5 Sharpened – Cactus-sharpened.ppm

3.2 Modifications

I was unable to get the GIMP tool working because of an NVIDIA library conflict. So, I resulted to using ffmpeg to convert the JPEG image to a PPM file. Once the file was in the correct input format, I noticed that the sharpen tool had hardcoded the image width and height. These did not align with the Trees image. So, I added a dynamic method to parse the height and width of the

image using the header and sscanf. This was a relatively simple change and once the height and width of the image were parsed, the rest of the code worked out of the box.

Changes:

1. Increased header buffer size. This was done to get the provided Cactus image to work with sharpen.
2. Add dynamic sizing, if possible, for standard ppm file format (no comments) – default to 400x300
3. Add error messages if read/write functions fail

3.3 sharpen-grid

The `sharpen_grid` utilizes threads to improve the execution speed of the process. It also splits the image into smaller slices, giving one to each thread. It utilizes the coordinate location of the pixel, row by column, to compare it to the pixels around it. This is a better method for sharpening images because of the parallelism introduced with threads and the special locality, which caching can use to improve performance, by using 2D arrays (coordinates). However, there are many silent issues with `sharpen_grid`. First, there is a lot of hardcoded values. The code only properly uses 12 threads out of the 48 that are created. Threads with `thread_idx >= 11`, are repeating the same work. Also, the size of the image is hardcoded to be a very large 4000x3000 pixel image. This is much larger than what we are using and wastes a lot of memory and process time. In fact the `sharpen_grid` application is significantly slower than the `sharpen` application for these reasons.

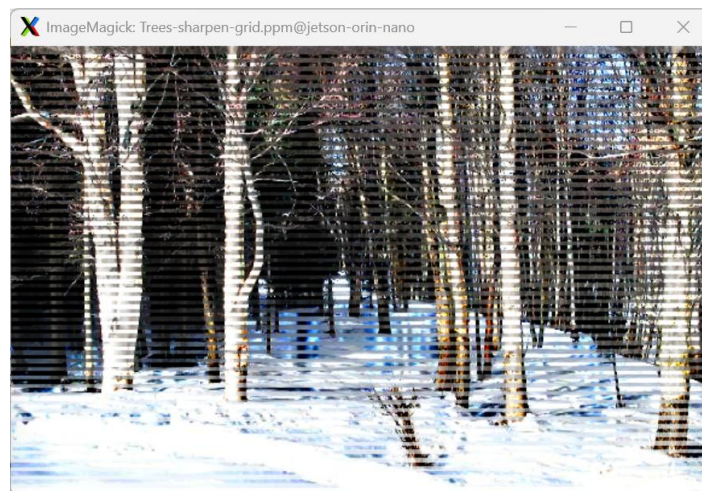


Figure 3.6 Sharpen-grid – Trees-sharpen-grid.ppm

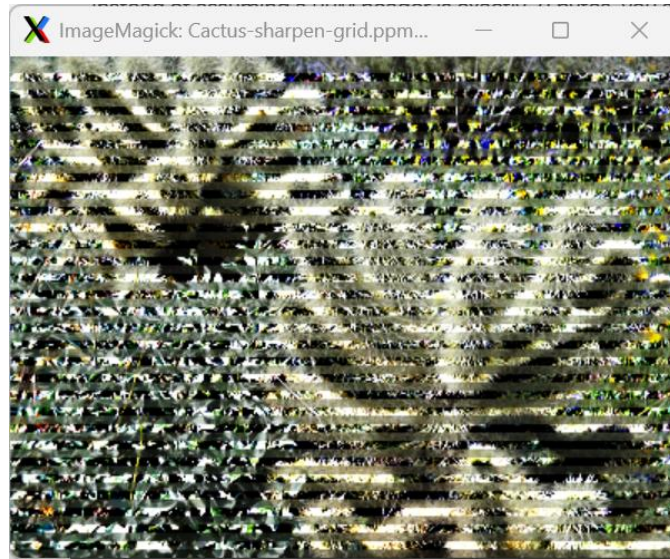


Figure 3.7 Sharpen-grid – Cactus-sharpen-grid.ppm

As seen in the figures above, there is a distinct grid like pattern overlaying the new images. This pattern is a result of the inadequate use of threads and hardcoded sizes in the application source. In order to improve this, the developer would need to introduce dynamic thread creation.

4 Face Detection

The Haar Cascades used in the Viola-Jones object detection algorithm in OpenCV 4.8.0 allow computers to detect human faces in real-time on very limited hardware.

The algorithm finds patterns by considering regions, as opposed to individual pixels. It uses black and white rectangles to divide the regions. Then, in order to improve the efficiency of the calculations, the image is converted into an integral image. In an integral image, any point is the sum of pixels to the left and above it. This allows the sum of the pixels inside any size rectangle to be calculated with just four pixel lookups. Next, the algorithm uses the AdaBoost machine learning algorithm to train on over 160,000 Haar-like features. This algorithm uses positive and negative images to find features that have a low error rate and down select the features to roughly 6,000 weak classifiers. Together, these weak classifiers form a strong classifier for the Adaboost algorithm to train and evaluate on. The last speed-up effect is the Cascade of Classifiers. This groups the features into stages which have various features. If a region passes all 38 of the stages, it is considered a face region.

OpenCV provides a training model and pretrained Haar Cascade models to detect faces and eyes in an image. The pretrained models we used detect a face. If there is a face, it draws an ellipse around the face and then detects eyes within the face. If an eye is detected, an ellipse is drawn around it. Observe the example in Figure 4.1 which detected the face, but only one eye on Professor Siewert.

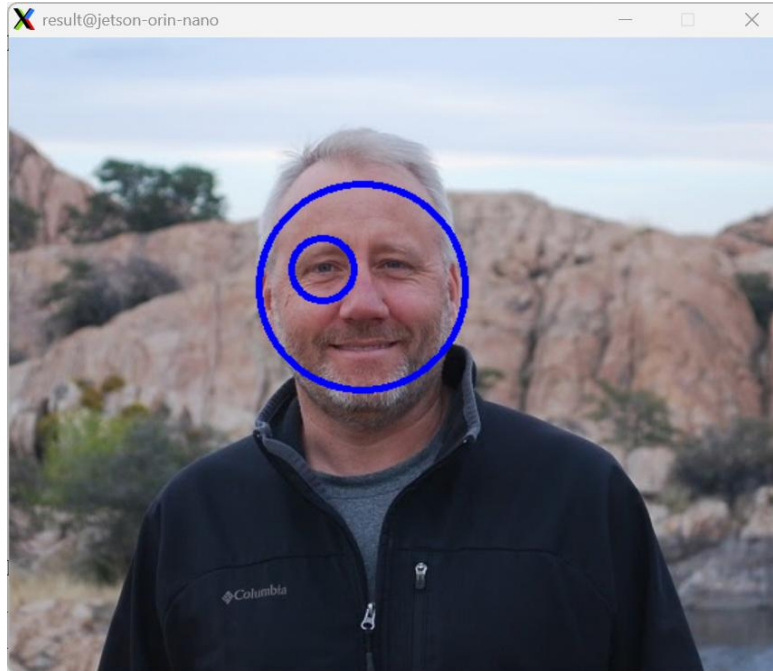


Figure 4.1 Face detection of sam_pic.jpg

This image was pulled from the course website. The file is 571x469 pixels. The facedetect application reported a detection time of 70.017ms.



Figure 4.2 Face detection of Sam-Headshot.jpg

Figure 4.2, above, was pulled from the official CU faculty website. It originally downloaded as an AVIF file. Using CloudConvert, I converted it into a JPEG of size 1500x1465 pixels. The facedetect application reported a detection time of 390.068 ms.

To run the facedetect application, I had to copy over the updated source code that comes available with OpenCV 4.8.0. Also, I made updates to the Makefile to correctly include the C flags and libraries for OpenCV.

5 Annotated Image

As seen in Figure 5.1 below, the `annotated_image` application captures a 320x240 pixel resolution image before adding a four (4) pixel yellow perimeter border and a one (1) pixel yellow cross hair through the center row and center column of the frame. The modified frame is displayed in a named window called “`annotated_window`.” The window continuously updates with each frame captured by the C920X web camera.

To generate the yellow perimeter border around the captured frame, the application calls the OpenCV `copyMakeBorder()` API. This creates a new frame with the border applied, enlarging the final frame to be 328x248 pixels. After generating the border, the application used the OpenCV `line()` API to draw the one (1) pixel width center crosshairs. The application used the original `WIDTH` and `HEIGHT` macros to calculate the start and stop points of the two lines used to create the crosshairs. The `line()` API does not generate a new frame. Instead, it rewrites each pixel along the line specified by the caller.



Figure 5.1 `annotated_video` – capture with perimeter and crosshairs

6 References

- [1] S. Siewert, "Cloud scaling, Part 3: Explore video analytics in the cloud - Using methods, tools, and system design for video and image analysis, monitoring, and security," IBM Developer, 2013.
- [2] CloudConvert, "AVIF to JPG Converter." [Online]. Available: <https://cloudconvert.com/avif-to-jpg>.
- [3] University of Colorado Boulder, "Sam Siewert." [Online]. Available: <https://www.colorado.edu/ecee/sam-siewert>.
- [4] University of Colorado Boulder, "ECEN 5763 Course Website." [Online]. Available: <https://sites.google.com/colorado.edu/ecen5763>.
- [5] Stack Overflow, "Find OpenCV version installed on Ubuntu." [Online]. Available: <https://stackoverflow.com/questions/8804064/find-opencv-version-installed-on-ubuntu>.
- [6] OpenCV, "OpenCV 4.8.0 Documentation." [Online]. Available: <https://docs.opencv.org/4.8.0/index.html>.
- [7] OpenCV, "Cascade Classifier Tutorial." [Online]. Available: https://docs.opencv.org/4.8.0/db/d28/tutorial_cascade_classifier.html.

7 Appendix

GitHub Repo: https://github.com/mamc3334/ECV_5763/tree/ex1

Pull Request: https://github.com/mamc3334/ECV_5763/pull/1

Forked ECV Repo: <https://github.com/mamc3334/ECV-ECEE-5763-mcgaffin/tree/ex1>

Pull Request: <https://github.com/mamc3334/ECV-ECEE-5763-mcgaffin/pull/1>

Build Environment

Platform: NVIDIA Jetson Orin Nano

OS: Ubuntu 22.04.5 LTS

Compiler: g++ 11.4.0

GNU Make 4.3

OpenCV Version: OpenCV 4.8.0